DBMS SE Unit -I

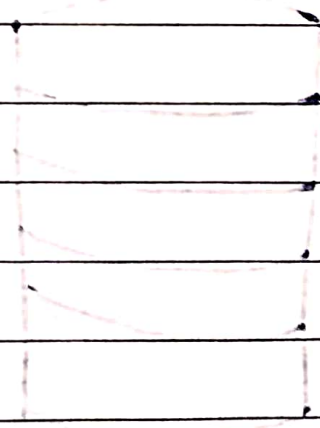
M T W T F S S
Page No.:
Date:
YOUVA

Syllabus:

Introduction, Purpose of DBMS. Database Applications, Data Models, DBMS Architecture, DBMS languages.

Database Design and ER Model: Entities, Attributes, Relationships, Keys constraints, ER Diagrams, Design process, Design Issues. Extended ER Features, Converting ER/EER diagrams to relational tables.

Case Study: Analise and design a database. (Using the ER model) for a real time Applications and convert it to relational table



Introduction of DBMS

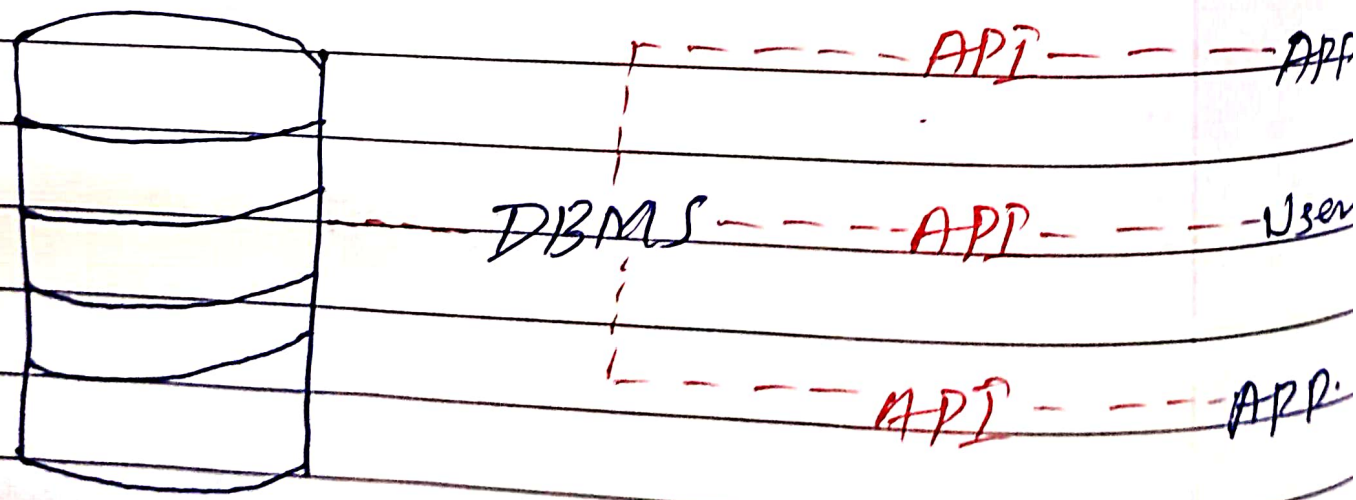
DBMS (Database Management System) is a software system that manages, stores and retrives data efficiently in a structure format.

* It allows users to create, update, and query databases efficiently.

* Ensure data intigrity, consistency and security across multiple users and applications.

* Reduces data redundancy and inconsistency through centralized control.

* Supports concurrent data access, transaction management, and automatic backups.



DBMS act as bridge between a central database and multiple

Clients - including apps and users.

It uses API's to handle data requests. enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional file-Based Systems. / Purpose of DBMS.

Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. Whèle this approach allowed users to store, retrieve and update file as needed, it came with numerous challenges.

* Data Redundancy
* Inconsistency
* Difficult Access
* Poor Security
* Single - User Access

* No Backup / Recovery :-

A university file-based system stori... data in seperate files (e.g. Academi... Results, Hostels) often faced this prob...

Components of Applications

1. Hardware
2. Software
3. Data
4. Procedures
5. Database Access language
6. People

① Hardware :-

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer)
- Stores and processes data; interfaces between real world input and digital systems.

② Software

- Actual DBMS software like MySQL, Oracle, PostgresQL
- Includes the database engine, OS, network software, & application tools.
- Transistors database access languages into operations.

⑤ Data

- Raw facts stored in structured or unstructured formats.
- Operational Data: Actual user data (eg. name, age)
- Metadata: Data about data (eg. storage time, size)
- Core reason DBMS exists - to manage & store data efficiently.

⑦ Procedures:

- Instructions & rules for using DBMS effectively.
- Covers setup, login/logout, data validation, backup access control, & report generation.

⑧ Database Access language

- Used to interact with the database
- Ex: SQL, My access, PL/SQL.
- DDL (Data Definition language) - Create, ALTER, DROP
- DML (Data Manipulation language) - INSERT, UPDATE

⑥ People:

- Users interacting with DBMS at different levels
- DataBase Administration (DBA) - Manage security, user access.
- Developers: Build applications using the database
- End Users: Use Application to access the database

Types of DBMS

1. Relational Database Management system (RDBMS)
2. NoSQL DBMS
3. Object-Oriented DBMS (OODBMS)
4. Hierarchical Database
5. Network Database
6. Cloud-Based Database

1. **Relational database management system**

- It organizes data into tables (relations) composed of rows & columns.
- Uses primary keys to uniquely identify rows and foreign keys to ~~uniquely~~ establish relationships between tables

2. **NOSQL DBMS**

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive
- They store data in various non relational formats, such as key-value pairs, documents, graphs or columns.

M T W T F S S
Page No.:
Date:
YOUVA

3. **Object oriented DBMS**

- It integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects.
- Supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and real-world simulations.

4. **Hierarchical Database**

- Organizes data in a tree-like structure where each record (node) has a single parent and have multiple children.
- This model is similar to a file system.

5. **Network Database**

- It uses a graph-like model to allow more complex relationships between entities.
- Data is represented using records and sets, where sets define the relationships.

6. **Cloud-Based Database**

- They are hosted on cloud computing platforms like AWS, Azure & Google cloud.
- These database can be related (NoSQL) and are maintained by cloud service providers, reducing administrative overhead.
- Example: Amazon RDS, MongoDB

* Database Languages.

Database languages are specialized sets of commands and instructions used to define, manipulate and control data within a database. Each language type plays a distinct role in database management, ensuring efficient storage, retrieval and security of data. The primary database language includes.

① Data Defination language (DDL)

* CREAT
* DROP
* TRUNCATE
* RENAME
* ALTER

② Data Manipulation Language (DML)

* INSERT
* DELETE
* UPDATE

③ Data Control language (DCL)

* REVOKE
* GRANT

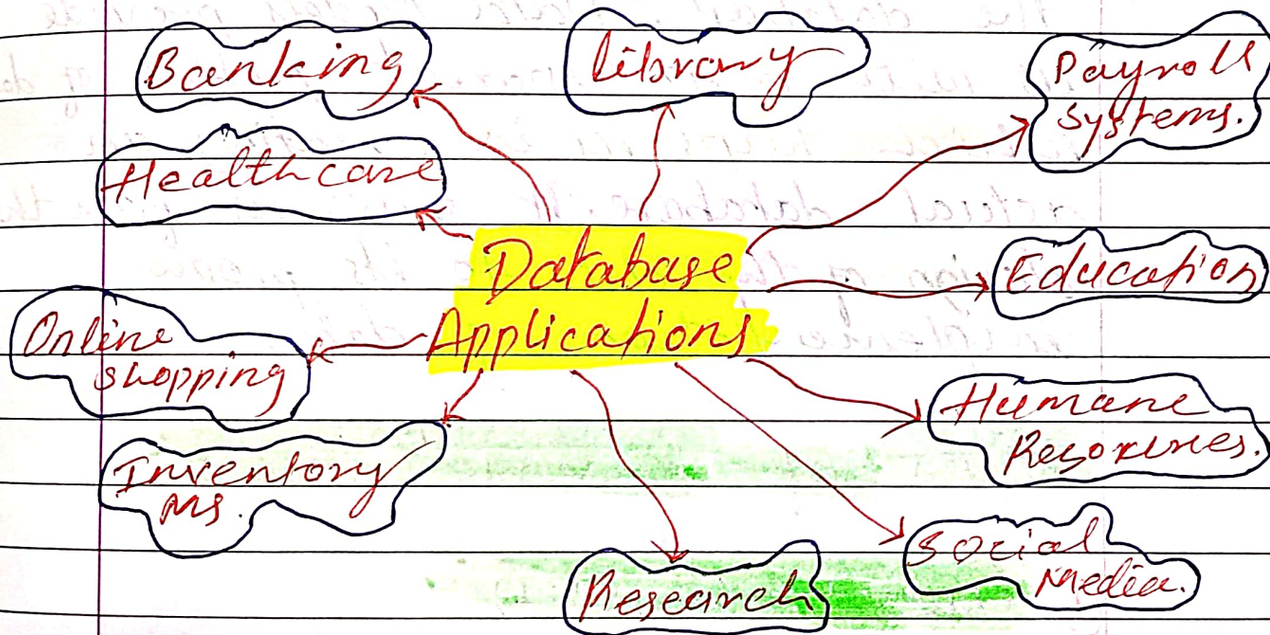
④ Transaction Control language (TCL)

* ROLLBACK
* COMMIT

⑤ Data Query Language (DQL)

* SELECT.

* Applications of DBMS



Banking: - Manages accounts and transactions.

E-Commerce: - Tracks products, orders and customers.

Healthcare: - stores patient records and diagnoses.

Education: - Handles student grades and schedules.

Social Media :- Manages user profile and interactions.

Data Science :- Supports analytics and predictions.

* Data Models

A Data Model in DBMS is the concept of tools that are developed to summarize the description of the database. Data Models provide us with a transparent picture of data which helps us in creating an actual database. It shows us from the design of the data to its proper implementation of data.

Types of Relational Models

① (One - to - One) (1:1)

- One record in table A is related to only one record in table B, and vice versa.

② (One - to - Many)

- One record in table A can be related to many records in table B, but each record in B relates to only one in A

③ Many - to - One

- Many records in table A are related to one record in table B.
- This is the reverse of one to many

④ Many - to - Many

- Many records in tables A relate to many records in table B.
- Implemented using a junction (bridge) table.
- Example :- Students <→ Courses.

* Types of Data Models.

1. **Hierarchical Data Models**
   - Data is organized in a tree structure
   - Each parent can have many children, but each child has only one parent.
   - Uses one - to - many relationships.

2. **Network Data Model.**
   - Data is organized as a graph.
   - A child can have multiple parents.
   - Support many - to - many relationships.

3. **Relational Data Model.**
   - Data is stores in tables (relations)
   - Relationships are maintained using primary and foreign keys.

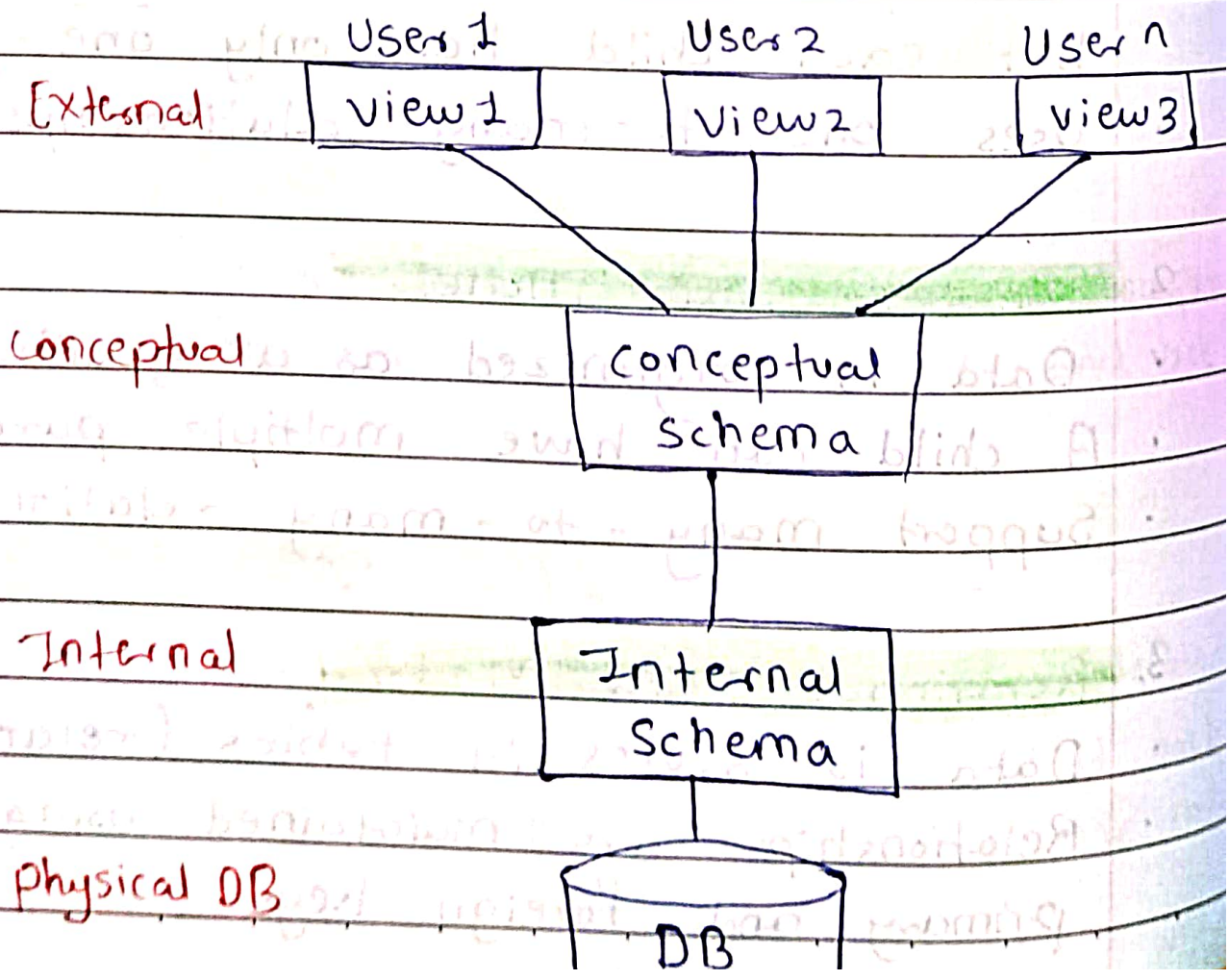
4. **Entity - Relationship (ER) Model**

- Conceptual model used for database design.
- Uses entities, attributes, and relationsh
- Represented using ER diagrams.

5. **Object - Oriented Data Model**

- Data ~~is~~ ~~Store~~ and their relationshi are contained in a single structur which is referred to as an object in the data model.
- It supports classes, inheritance, encapsulation.
- It is a combination of OOP & RDBMs

* **DBMS Architecture**



* Components of 3-Tier Architecture.

1. **Presentation Tier** (User Interface Layer)
- The presentation tier is the user Interface or client layer of the application.
- It ensures the application is user-friendly by providing interface such as web browsers, mobile apps or desktop applications.

2. **Application Tier** (Business Logic Layer)
- It acts as the intermediary between the presentation Tier and the Data Management Tier.
- It is responsible for processing and managing the business logic of the application.
- It communicate with the presentation tier to receive user input & communicate with data management tier to retrieve or store data.
- This Tier include application servers, web servers or APIs.

3. **Data Management Tier** (Database Layer)
- It is a Bottom layer of the 3-tier architecture.
- It is responsible for managing and storing data.

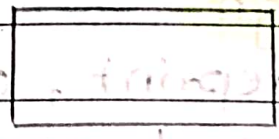
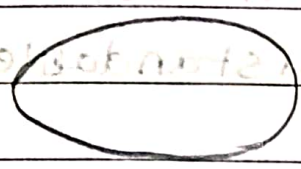
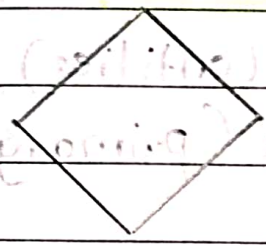
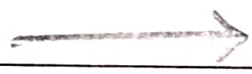
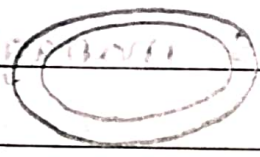
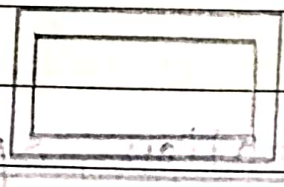
- It ensure that the data is safely stored retrieved and maintained.
- It handles the database connections and ensures data integrity.

* ER Model in Database Design.

- The Entity - Relationship Model is a conceptual model for designing a database.
- This model represents the logical structure of a database, including entities, their attributes, and relationships between them.

- **Entity**: An object that is stored as data.
  e.g Student, Course or Company
- **Attribute**: Properties that describe an entity
  e.g StudentID, CourseName or EmployeeEm
- **Relationship**: A connection between entities.
  e.g Student enrolls in Course.

Symbols used in ER Model

Symbols	Representation
	Entities
	Attributes
	Relationships
	Connection
	~~Double~~ Multi-Valued Attribute
	Weak Entity

Design Process

Follow the steps for designing a database for an application.

- Gather the requirements (Functional and data) by asking questions to the users.

- Create a logical or conceptual design of the database. This is where ER model plays a role.

Step 1: Conceptual Design :-
It serves as the blueprint, capturing data requirements and business rules in a universally understandable format

Step 2: Stouture Definition :-
Helps defines tables (entities), columns (attributes), and keys (primary / Forgein keys).

Step 3: Implementation :-
Translates into relational database tables, with relationships managed via foreign key.

Step 4: Refinement :-
Concepts like Generalization, specializatio and Aggregation are used to manage complexity for longer datasets.

M T W T F S S
Page No.:
Date:
YOUVA

* Design issues

1. **Entity set vs. Attribute:**
Deciding if something (like a phone number) should be an attribute of an entity or its own entity set.

2. **Entity set vs. Relationship set:**
Determining if an object is better modeled as a standalone entity or as a connection (relationship) between other entities.

3. **Binary vs. N-ary Relationships:**
Choosing between simple two entity (binary) relationships or more complex relationships involving three or more entities (n-ary).

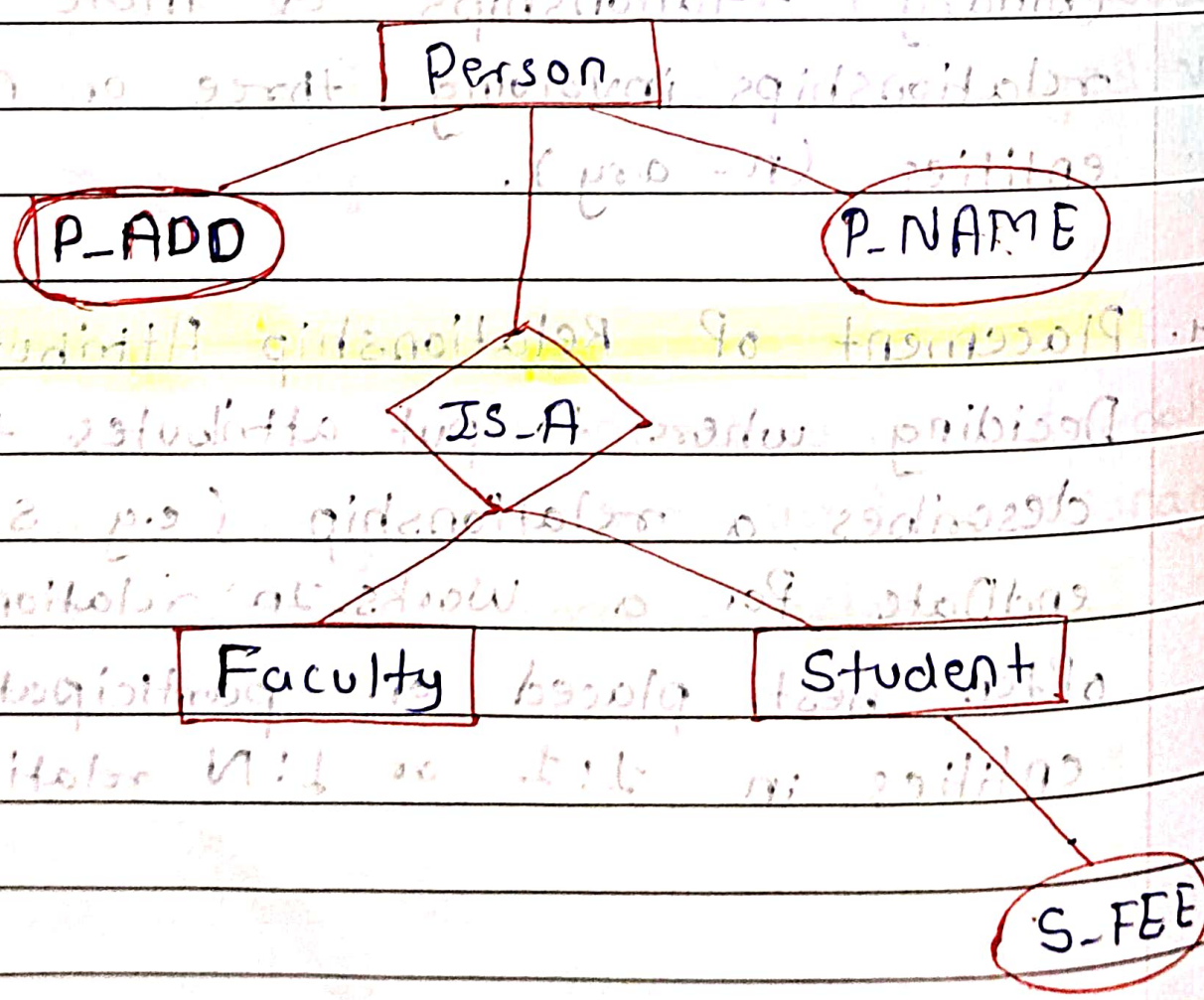
4. **Placement of Relationship Attributes:**
Deciding where to put attributes that describes a relationship (e.g startDate, endDate for a Works_In relationship) often best placed on participating entities in 1:1 or 1:N relationship.

* Extended ER (EER) Features :

1. Generalization
2. Specialization
3. Aggregation

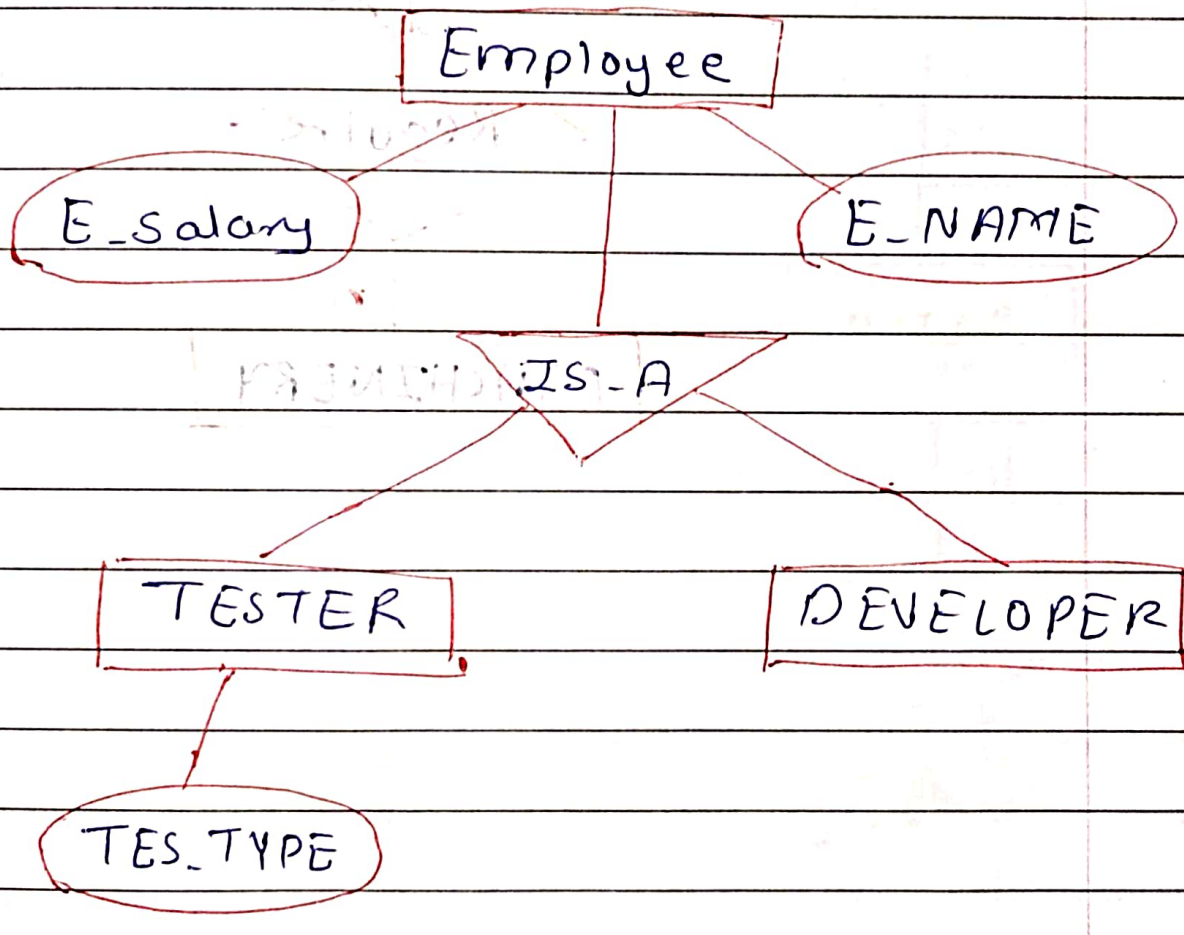
1. Generalization :-

- Generalization is the process of extracting common properties from a set of entities and creating a generialized entity from it.
- It is a bottom-up approach in which two or more entities can be generialized to a higher-level entity if they have some attributes in common.



2. **Specialization** :-

- In specialization, an entity is divided into sub-entities based on its characteristics.
- It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities.

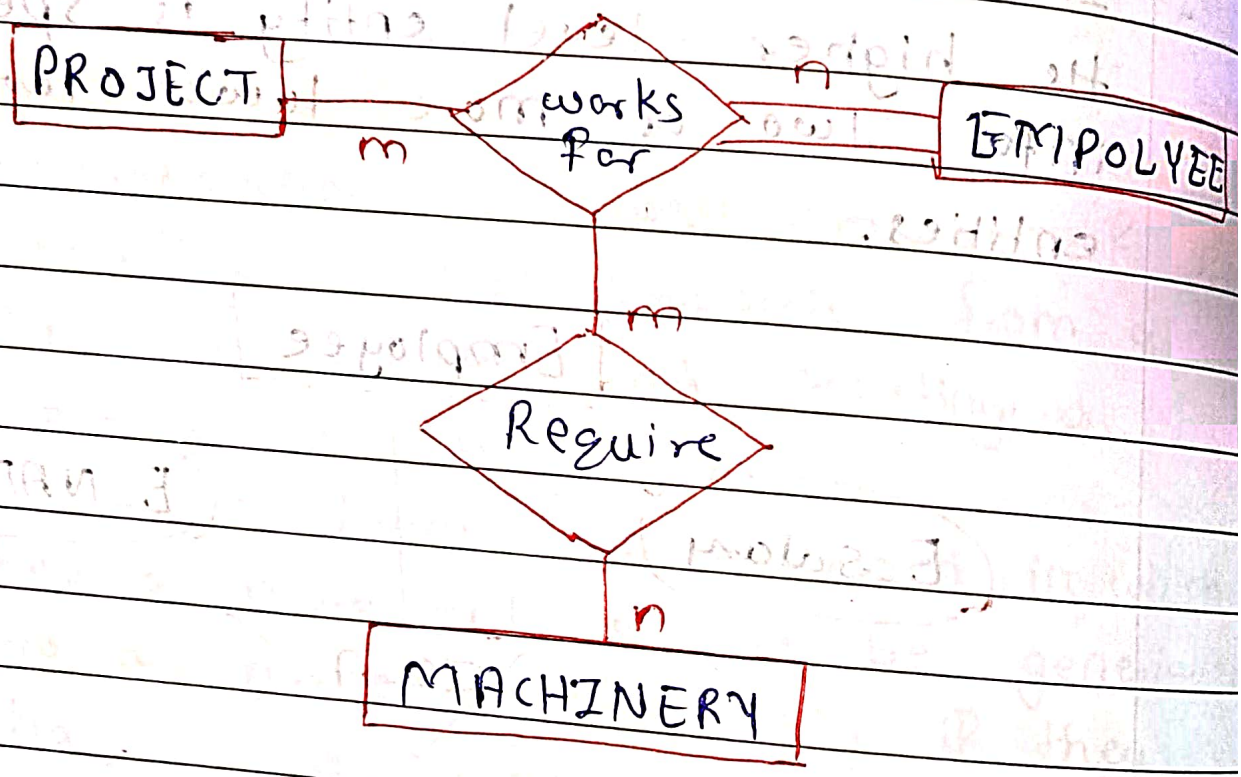


3. **Aggregation** :

- An ER diagram is not capable of representing the relationship between entity and a relationship which may be required in some scenarios.
- In those case, a relationship with its corresponding entities is aggregate

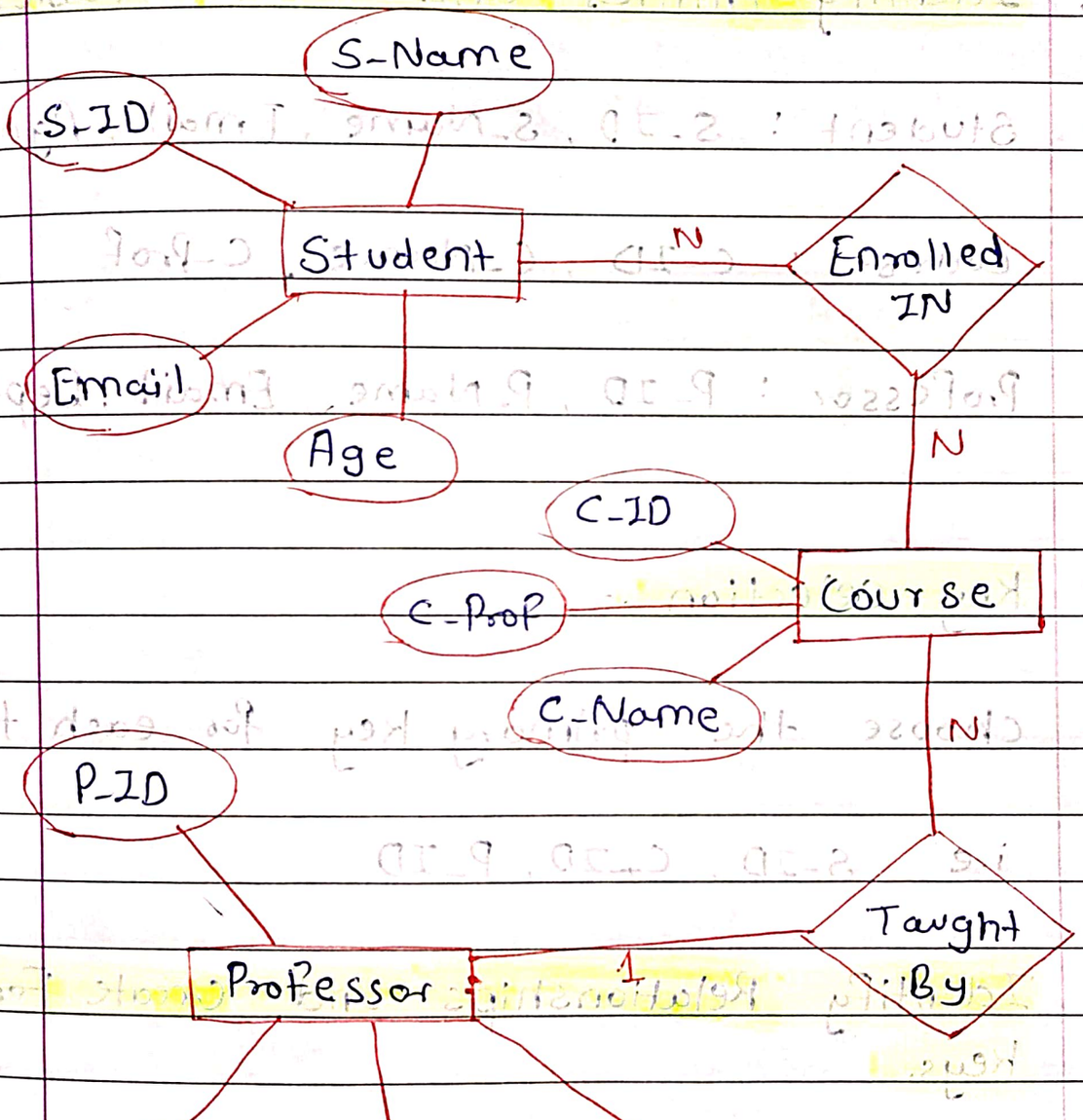
into a higher-level entity.

→ Aggregation is an abstraction through which we can represent relationship as higher-level entity set.



* ER Diagram

College database system :



* Convert the ER Diagram to relation table.

1. Identify Entities and Create Tables :-

- Student : S_ID, S_Name, Email, Age
- Course : C_ID, C_Name, C_Prof
- Professor : P_ID, P_Name, Email, Dept.

2. Key selection :-

- Choose the primary key for each tabl

  i.e S_ID, C_ID, P_ID

3. Identify Relationships and Create Foreig keys.

- If entities have relationships, break them down and reduce the table if possible.
- Relationships in the ER diagram are represented by Foreign keys in the relational model.

- **Final Relational model looks like :**

- Student : S_ID (PK), S_Name, Email, Age
- Course : C_ID (PK), C_Name, C_Prof (FK)
- Professor : P_ID (PK), P_Name, Email, Dept.
- Enrollment : S_ID (FK), C_ID (FK),

Student

S_ID	S_Name	Email	Age
S1	Arsh	abc	18
S2	Ram	xyz	19
S3	Alia	PQR	18

Course

C_ID	C_Name	C_Prof
C1	OS	P_1
C2	DBMS	P_2
C3	OOP	P_3

Proffesor

P_ID	P_Name	Email	Dept
P_1	Aniket	jhg	CES
P_2	Tushar	asd	IT
P_3	Rahul	xop	AI & DS

Enrollement :

C_ID	C_Name	C_Prof
C1	OS	P_1
C2	DBMS	P_2
C3	OOP	P_3